

FIT2109 Computer Science Workshop

Week 3: Editor / Modern IDE

Zahra Mirzamomen

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Why this workshop matters

This week's reading gave us the concepts

Buffers, modes, navigation, refactoring, debugging, remote editing — as ideas you can explain.

This workshop is about doing it

Today is keystrokes, not definitions: buffers & modes → editors → tooling → navigation → multi-file editing → debugging → remote.

Learning goals

By the end of this workshop, you should be able to:

1. Explain what an editor manages: files, buffers, modes, and selections.
2. Understand how to work efficiently in Vim and VS Code.
3. Navigate an unfamiliar project by name, symbol, and text search.
4. Choose the right tool for a repeated change: direct edit, multi-cursor, search-and-replace, or refactor.
5. Explain why editors support remote development, and find an entry point in code you didn't write.

Recap: editor concepts

The big idea from the pre-reading

An editor is the interface between your intentions and a program's source: what you're changing lives in memory first, and only becomes real when you save. Every editor — however it looks — is built around that same idea.

Why it matters in the job

Professionals spend far more time reading and modifying existing code than writing new code from a blank file. Fluency with your editor isn't a nice-to-have — it's what lets you spend your attention on the problem instead of on the tool.

Today's goal

Turn what you can already explain into something your hands do without thinking.

Two families of editors

Terminal-based

Runs inside a terminal, no mouse required.

Popular names: Vim, Nano, Emacs.

GUI-based

A full graphical application with panels, mouse, and extensions.

Popular names: VS Code, JetBrains IDEs.

Today we go deep on two of these: Vim, for when a GUI isn't available, and VS Code, the GUI editor you'll likely use day to day.

You won't always have VS Code

Terminal editors matter because...

-  **Remote servers** — SSH into headless Linux machines.
-  **Containers & cloud** — minimal images often ship only `vi/vim`.
-  **Production support** — a fast fix when a service is down.
-  **Reliable over slow networks** — no heavy GUI to drag over SSH.
-  **A real professional skill** — common in SWE, DevOps, sysadmin, and cybersecurity.

Goal

You don't need to master Vim — you need to be able to confidently make small edits when it's the only editor available.

Choosing a terminal editor: why Vim

nano

Easiest to start — shortcuts shown on screen. Fine for a one-line edit, but limited beyond that.

Vim

Modal editing takes practice, but it's fast, scriptable, and installed almost everywhere.

Emacs

Extremely powerful and extensible, but a much bigger investment to learn.

Why Vim, specifically

It (or its ancestor `vi`) ships by default on almost every Unix-like system. Unlike nano's convenience or Emacs's depth, Vim's real advantage is that it's *always there* when you need it.

Vim survival skills

By the end of today, you should be able to:

- ✓ Open a file
- ✓ Understand Normal vs Insert mode
- ✓ Navigate efficiently through a file
- ✓ Search for text
- ✓ Edit text (insert, delete, replace)
- ✓ Copy and paste (yank & put)
- ✓ Undo and redo changes
- ✓ Save changes
- ✓ Exit Vim (with or without saving)

That's it

If you can do these tasks, you can comfortably survive in Vim.

Vim basics

Watch

- `vim notes.txt` opens the file.
- `i` for Insert, `Esc` back to Normal.
- `:w` saves, `:wq` saves and quits, `:q!` quits without saving changes.

Handout Activity 1: Getting comfortable in Vim (10 minutes)

Now work on Activity 1 in the handout

Create a new file: `vim newfile.txt`. Write a few lines, then save and quit (`:wq`). Reopen the same file, add more to it, and save and quit again.

Play around

Keep it simple: open a file, make a basic edit, then either save and exit (`:wq`) or quit without saving (`:q!`). That's the whole loop for this activity — we'll build on it from here.

Live demo: Vim survival (continue)

Watch

- Navigating through a file.
- `/word` + Enter searches; `n` repeats it.
- Copy and paste a line/block.
- Motions combine with operators to move and edit precisely.
- `u` undoes, `Ctrl+r` redoes.

Note

You're ready for Handout Activity 2.

Handout Activity 2: Vim survival (20 minutes)

Now work on Activity 2 in the handout

Open `vim-mission-sysinfo.sh` **in Vim**. Work down through the numbered missions in the file's comments, in order — after each one, save, exit, and run the script to see the effect, then make a backup, reopen the script again for the next mission.

Compare notes around the table

Work through the missions individually, on your own machine. Then compare with the people at your table — how did you do each one, and who found a more efficient way?

Why VS Code fluency matters

Vim gets you unstuck — VS Code is where you'll live

Most day-to-day development happens in a full GUI editor, not a bare terminal. The efficiency habits you build from here on are the ones you'll use every single day.

Why it matters

Professionals spend far more time navigating, searching, and modifying existing code than typing new code from scratch — a slow editor workflow costs real time, every day, for years.

From here

We go deep on VS Code in two parts: navigating a project and reading unfamiliar code, then choosing the right tool to refactor it.

VS Code: Basics

Key idea

- **Open a folder** (*File > Open Folder*, or `code .`) — not just a single file.
- **Tabs & editor groups** — every open file is a tab; groups let you split the window into panes.
- **Direct editing** — no mode to switch between typing and commands, unlike Vim.
- **Integrated terminal** — a full shell built into the same window.
- **Command Palette** (`Ctrl` + `Shift` + `P` on Windows/Linux, `Cmd` + `Shift` + `P` on Mac, or `F1` on either) — every action is a named, searchable command.

Shortcuts in this deck: Windows/Linux first, Mac second — `Ctrl` ↔ `Cmd`, `Alt` ↔ `Option`.

Live demo: VS Code basics

Watch

Getting started in VS Code — opening a folder, creating a file, editing it, and saving.

VS Code: Tooling

Key idea

- VS Code is an editor connected to **language tooling** — some ships built in, richer support comes from an extension.
- **Built in:** JavaScript, TypeScript, JSON, HTML, CSS.
- **Needs an extension:** Python, Java, C/C++, and most other languages.
- Hints range from squiggly **diagnostics** to **Quick Fixes** (Code Actions) that fix things for you — like organizing imports automatically on save.

Formatting

"How should this look?" Layout, spacing, indentation — style only.

Linting

"Does this look suspicious?" Unused imports, unreachable code — logic hints.

Live demo: tooling

Watch

Call a function with the wrong number of arguments — watch the squiggle appear. Introduce an unused variable, then format the file with `Format Document`.

VS Code: Navigating a project

Key idea

- The **Explorer** shows your project's real folder structure.
- Quick Open (`Ctrl` + `P` / `Cmd` + `P`) jumps to a file by name.
- Find in Files searches a whole workspace.
- **Go to Definition** and breadcrumbs navigate by meaning, not just by file path.

Live demo: navigate & search

Watch

`Ctrl` + `P` (`Cmd` + `P`), type a partial filename, press Enter — that's Quick Open.
`Ctrl` + `Shift` + `F` (`Cmd` + `Shift` + `F`) searches a term across every file. Then follow a breadcrumb, or `Go to Definition`, to trace where something is called from.

Note

Search is often the first tool you reach for in a codebase you didn't write.

VS Code: Multi-File editing

Split editor

Compare files side by side. `Ctrl` + `\` (`Cmd` + `\` on Mac), or drag a tab to an edge.

Multi-cursor

- `Alt` + Click (`Option` + Click on Mac) — add a cursor where you click.
- `Ctrl` + `D` (`Cmd` + `D` on Mac) — select the next matching word too.
- `Ctrl` + `Shift` + `L` (`Cmd` + `Shift` + `L` on Mac) — select every match at once.
- `Ctrl` + `Alt` + `↓` / `↑` (`Cmd` + `Option` + `↓` / `↑` on Mac) — add a cursor on the line below/above.
- `Shift` + `Alt` + drag (`Shift` + `Option` + drag on Mac) — column (box) selection.
- `Esc` — back to a single cursor.

Live demo: split & multi-cursor

Watch

Split the editor (`Ctrl` + `\` / `Cmd` + `\`) to see a function beside its test. `Alt` + Click (`Option` + Click on Mac) to drop an extra cursor, or select a repeated variable and press `Ctrl` + `D` (`Cmd` + `D` on Mac) twice before typing a replacement.

Note

You can see every place that's about to change before you type — that's the whole point.

VS Code: Refactoring the code

Search & Replace

Edits raw text — blind to comments and strings. A workspace-wide replace can catch a false match inside a comment or test fixture, so always preview first.

Rename Symbol (**F2**)

Understands the program's structure and only changes real references. Needs the language's extension installed — without it, Rename Symbol isn't available.

Rule of thumb: reach for Rename Symbol first when renaming a code identifier; use Search & Replace for everything else.

Live demo: Rename Symbol

Watch

Try a workspace-wide search-and-replace first — notice it can catch a match inside a comment. Undo it, then use `F2` (Rename Symbol) on the same identifier instead.

Note

Same goal, different guarantees — Rename Symbol only touches real references.

Handout Activity 3: Navigating and Refactoring an Unfamiliar Codebase

Now work on Activity 3 in the handout

Explore a real Python project you've never seen — find the entry point, trace a bug report, compare files in a split view, fix a repeated value with multi-cursor, and refactor safely with Rename Symbol.

VS Code: Remote editing

Key idea

Remote-SSH, Containers, and WSL keep the editor interface local while the filesystem, terminal, and extensions run on the remote machine.

Not just convenience

This mirrors how real software gets built and debugged — on servers, in containers, in the cloud — not just on your own laptop.

Live demo: connecting to a remote host

Watch

Open the Command Palette → `Remote-SSH: Connect to Host`. A new window opens with the same interface, but the status bar shows you're now editing on the remote machine.

Exit ticket

Before next time

Keep practising both Vim and VS Code — the more these skills become automatic, the more attention you have left for the actual problem.

Exit ticket

Wrap up with the PollEv activity — we'd love to hear how today went for you!